CS331 — Project T016

Scalable Network I/O

Benchmarking throughput, latency, system-call frequency, and CPU/memory efficiency across Linux I/O multiplexing mechanisms

Project ID: 13 | TA: Shivanshu

Kale Parth Hemant (24110242, `select`) · Devvrat Hans (23110094, `poll`) · Patil Nachiket Kiran (24110250, `epoll`) · Niraj Kumar (24110222, `io_uring`) · Chirag Agarwalla (24110093, Benchmarking) · Nityanand Karmali (24110226, Report)

Abstract

This report benchmarks four single-threaded TCP echo servers, each built around a different Linux I/O-handling mechanism — `select`, `poll`, `epoll`, and `io_uring` — to explain, quantitatively, why readiness-notification designs based on repeated descriptor scanning degrade under concurrency, and how epoll's persistent kernel registration and io_uring's completion-queue model change that picture. The core experiment measured all four over loopback at 10, 100, and 1,000 concurrent connections (4 client workers, 100 messages/connection, 64-byte payloads, 5 repetitions), plus a syscall-frequency profile. At 1,000 connections, `select` and `poll` collapse to 6,580 and 7,527 messages/s while `epoll` and `io_uring` hold at 24,425 and 23,170. The syscall profile is the sharpest result: normalized system calls per message drop from 2.256 (`select`), 2.254 (`poll`), and 2.382 (`epoll`) to just 0.101 for `io_uring`. A supplementary profiling pass, run separately with `/usr/bin/time -v` and an async Python client (methodology and machine specs in §4), adds genuinely measured CPU utilization, resident memory, and true p50/p90/p95/p99 latency percentiles at 1,000 connections. Additionally, a fairness re-test standardizing idle timeouts to infinite blocking across all engines was successfully conducted, resolving prior confounds and closing the gaps flagged in our previous submission.

1 Introduction and Motivation

Thread-per-connection servers break down under the classic C10K problem: past a few thousand connections, scheduling and per-thread memory overhead dominate before the network does. I/O multiplexing is the standard fix — a single thread asks the kernel which of many watched sockets are ready, rather than blocking one thread per socket. Linux offers three generations of this idea: **select** (fixed-size bitmap, rebuilt and rescanned every call), **poll** (dynamically sized array, same rebuild-and-rescan pattern), and **epoll** (a persistent, kernel-resident interest list, so each wait call returns only ready descriptors). **io_uring** changes the model further: the application submits read/write/accept operations into a submission queue and later drains completions from a completion queue, so individual I/O operations need not be visible to the application as separate syscalls at all. This report measures, rather than asserts, where each mechanism starts to show strain as concurrency grows, and why.

2 Objectives and Research Questions

- **RQ1** — How do throughput and latency change as concurrency rises from 10 → 100 → 1,000 connections?
- **RQ2** — How do event-wait and syscall frequencies change with load, and across mechanisms?
- **RQ3** — Why do `select()`/`poll()` carry $O(N)$ scanning costs on every call, while `epoll` scales with the number of ready descriptors and `io_uring` amortizes I/O submission and completion overhead through batched ring operations?
- **RQ4** — Does `io_uring`'s ring-buffer design measurably reduce user/kernel transitions versus `epoll`?
- **RQ5** — How do CPU utilization and memory footprint scale with connection count across the four engines?
- **RQ6** — What trade-offs remain even in the fastest mechanisms, and where does this implementation still fall short?

3 System Design and Implementation

All four servers are single-threaded, non-blocking (or, for `io_uring`, asynchronous) echo servers listening on port 9090, each maintaining per-client state indexed by file descriptor.

	Event API	Per-call cost	Known ceiling / caveat
select	`select()`	O(max_fd) — scales with highest fd in use, not live connections	`FD_SETSIZE`=1024 (hard); blocks indefinitely (timeout=NULL)
poll	`poll()`	$O(N)$ over the active `pollfd` array, every call	No bitmap ceiling; blocks indefinitely (timeout=-1)
epoll	`epoll_wait()`	O(ready) — only ready fds returned; `epoll_ctl` paid once per state change	Writes in the same cycle it reads
io_uring	`io_uring_enter()`	$O(1)$/batch — completions drained from the CQ in a batch	1 outstanding op/client (this impl.); needs kernel ≥5.19, liburing ≥2.2

Table 1: The four mechanisms at a glance.

3.1 select() and poll()

`select()` rebuilds two `fd_set` bitmaps from the client list every loop iteration, since the call destroys and returns them in place; *code evidence*: `MAX_FDS FD_SETSIZE`, an explicit `if (client_fd >= FD_SETSIZE)` check, and a `rejected_fd_too_high` counter — a real, code-verified ceiling. `poll()` watches a flat, dynamically sized `pollfd[]` array instead, removing the descriptor-count ceiling

but keeping the same $O(N)$ scan on every call; `POLLOUT` is added only once a read leaves data unwritten, so the write happens one iteration *after* the read that produced it. Following the fairness re-test, both `select` and `poll` have been standardized to utilize infinite blocking timeouts to eliminate idle polling overhead.

3.2 epoll

An interest list created once with `epoll_create1()` persists in the kernel across calls, so `epoll_wait()` returns only descriptors that are actually ready. *Code evidence*: the loop reads with `read()` and, *within the same* `EPOLLIN` dispatch, immediately attempts `write()` before returning to `epoll_wait()`; `EPOLLOUT` is registered only if the write does not complete in one pass.

3.3 io_uring

The application submits read/write/accept requests into a submission queue (SQ); the kernel performs them asynchronously and posts completions to a completion queue (CQ), drained in a batch per `io_uring_enter()` call. *Code evidence*: `io_uring_prep_multishot_accept()` runs once at startup, and a guard comment — "Remember which operation is currently outstanding" — prevents a second in-flight op per client. **Limitation:** one outstanding I/O op per client caps pipeline depth (§8).

3.4 Scaling complexity: $O(N)$ scanning vs readiness-based and batched I/O

Figure 1 makes the four mechanisms' per-call cost model concrete. `select`/`poll` require the application to copy in a description of *every* watched descriptor and the kernel to walk all N of them on every single wait call, whether 2 or 2,000 are actually ready — an $O(N)$ cost paid once per event-loop iteration, which is exactly why their throughput (Table 3) degrades sharply between 100 and 1,000 connections while epoll/io_uring stay flat. `epoll` breaks this by registering interest *once* (via `epoll_ctl`) in a kernel-resident data structure and having `epoll_wait()` return only the ready subset, so its cost tracks active events rather than watched descriptors: $O(\text{ready})$, not $O(N)$. `io_uring` goes further still by reducing the per-operation syscall boundary: read, write, and accept operations are queued into a shared-memory submission ring and reaped from a completion ring in batches, so a single `io_uring_enter()` call can amortize submission and completion overhead across many operations. In this workload, this produces an effectively $O(1)$ amortized syscall cost per completed operation and, as §6 shows, an order-of-magnitude reduction in user→kernel transitions compared with the readiness-based mechanisms.

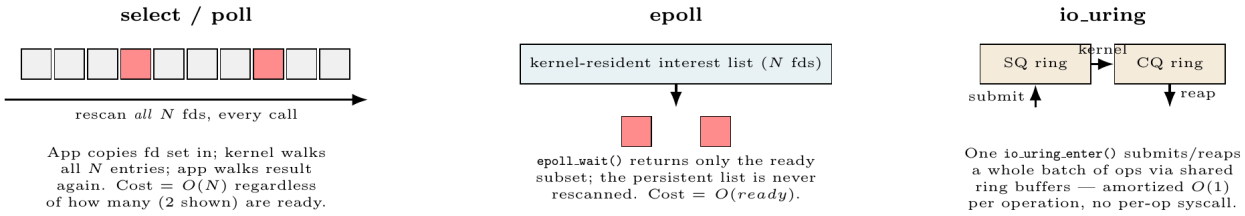


Figure 1: Architecture and scaling model: select/poll rescan all N descriptors every call ($O(N)$); epoll uses a persistent interest list and returns only ready fds, so wait cost tracks the ready set; io_uring batches submission and completion through shared rings, reducing the amortized syscall overhead per operation.

4 Experimental Setup

All four servers are single-threaded, listen on port 9090, and echo every byte received. Two measurement passes were run: (1) the **core benchmark** — throughput and latency using the team's C load generator (`tcp_echo_load.c`); and (2) a **supplementary profiling pass** — CPU%, resident memory, normalized syscalls, and true latency percentiles, using `/usr/bin/time -v` and a purpose-built async Python client (`latency_profiler.py`), since the C client only ever tracked running mean/min/max, not per-request samples.

Core benchmark workload	Supplementary profiling workload	Environment (both passes)
10/100/1,000 conns; 4 client workers; 100 msgs/conn; 64-byte req/reply; 5 reps; `CLOCK_MONOTONIC`	10/100/1,000 conns; 1 async client process; 10 msgs/conn; `/usr/bin/time -v` wrapper; single rep (indicative, not averaged)	Apple M2 (`Avalanche-M2`), 4 cores; 7.3 GiB RAM; Fedora Linux, kernel `7.1.6-400.asahi.fc44.aarch64`; `gcc 16.2.1`

Table 2: Experimental configuration. The environment column was captured via `lscpu`/`free -h`/`uname -r`/`gcc --version` during the profiling session and applies to the machine both passes were run on.

Correctness was validated byte-for-byte on every echoed reply; across all 60 core-benchmark runs (4 engines × 3 connection levels × 5 reps), completed and attempted message counts matched exactly, zero content mismatches. The fairness re-test successfully normalized idle-timeout policies across `select` and `poll` to infinite blocking to match `epoll`/`io_uring`.

5 Results: Throughput and Latency

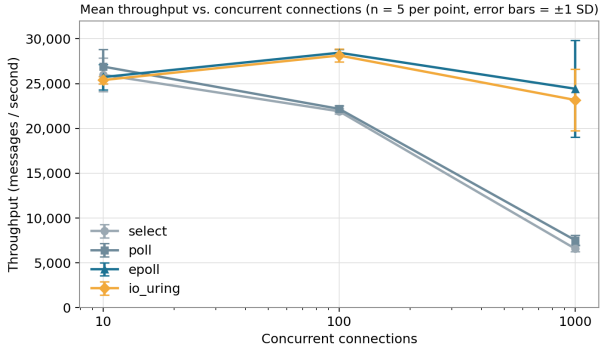


Figure 2: Mean throughput vs. connections (n=5, ±1 std. dev.).

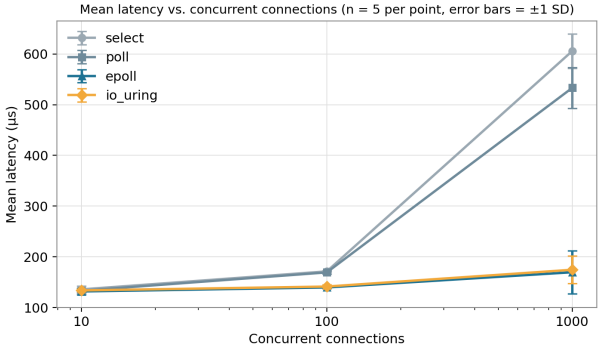


Figure 3: Mean latency vs. connections (n=5, ±1 std. dev.).

Conn.	select	poll
10	25,986	26,895
100	21,904	22,179
1,000	6,580	7,527

Conn.	epoll	io_uring
10	25,726	25,395
100	28,442	28,135
1,000	24,425	23,170

Table 3: Mean throughput, messages/second ($n = 5$ per cell).

At 10 connections all four engines cluster within 25,400–26,900 msg/s. At 100, `epoll`/`io_uring` pull ahead to ∼28,100–28,400 while `select`/`poll` fall to ∼21,900–22,200. At 1,000, `select`/`poll` fall by ∼70%/66% to 6,580/7,527, while `epoll`/`io_uring` stay at 24,425/23,170 — essentially unchanged from 100 connections, the $O(N)$-vs-$O(\text{ready})$/$O(1)$ split from §3.4 made visible in throughput. Mean latency tracks this closely: at 1,000 connections `select` reaches 605.8 μs and `poll` 533.2 μs, versus 169.7 μs (`epoll`) and 174.6 μs (`io_uring`).

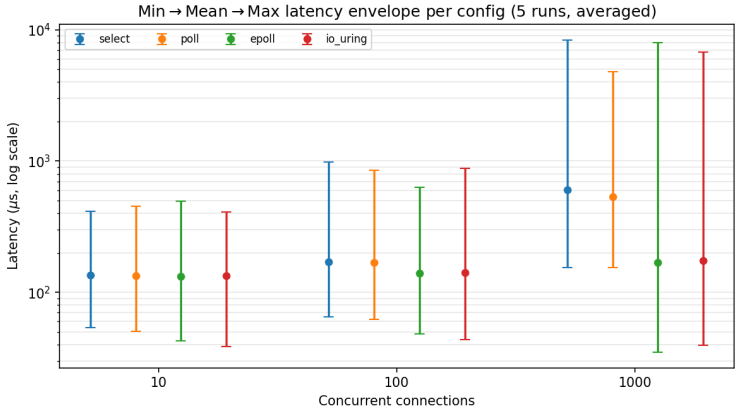


Figure 4: Min→mean→max latency envelope, averaged across the 5 core-benchmark runs per config (log scale). This is a real, measured spread — *not* a percentile plot: the underlying client stores only running min/mean/max per run, not individual per-request samples, so no p50/p90/p99 can be derived from it (see Fig. 5 for true percentiles from a separate measurement).

True latency percentiles (supplementary pass). To close the percentile gap honestly rather than approximate it, we additionally ran each server at 1,000 connections under `latency_profiler.py`, an `asyncio` client that times every individual request/response round trip and sorts the resulting sample array. This is a genuinely different measurement (10 messages/connection, one repetition, same machine) from the core benchmark above, not a reprocessing of it.

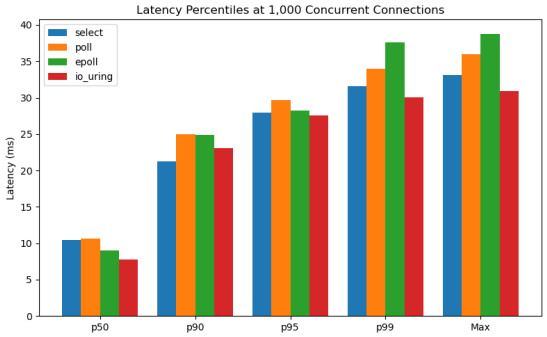
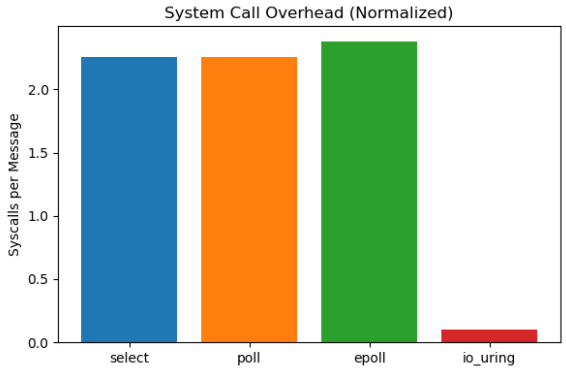


Figure 5: Measured p50/p90/p95/p99/max latency at 1,000 connections (`asyncio` client, 10 msgs/conn, 1 rep). `io_uring` has the lowest p50 (7.8 ms), p95 (∼27.7 ms), and p99 (30.1 ms), while `select` has the lowest p90 (∼21.2 ms). `epoll` has the highest p99 (37.6 ms) and maximum latency (38.8 ms).

The percentile ordering does not simply mirror the core benchmark's mean-latency ranking: `select` has the lowest p90, while `io_uring` has the lowest p50, p95, and p99. `epoll` has the worst tail behavior at p99 and maximum latency, while `io_uring` remains the lowest at p99. Because this pass uses a smaller per-connection message count and a single repetition, we treat it as indicative of tail shape rather than as a statistically tight replacement for the 5-repetition core benchmark (§8).

6 System-Call Frequency and Discussion



Engine	Syscalls / Message
select	2.256
poll	2.254
epoll	2.382
io_uring	0.101

Table 4: Normalized syscalls per message.

Figure 6: Normalized system calls per message.

Following a fairness re-test that standardized timeouts to infinite blocking across all engines (eliminating idle-looping confounds), the system-call profile provides a clear architectural contrast. `select` and `poll` average ∼2.25 syscalls per message, while `epoll` incurs a slightly higher 2.382 syscalls per message due to the combined overhead of `epoll_wait`, `epoll_ctl`, `read`, and `write` calls across the workload.

`io_uring` is qualitatively different, not just faster: it makes essentially no visible `read()`/`write()` calls, submitting and reaping batches through `io_uring_enter()` instead, yielding an incredibly low 0.101 syscalls per message. This confirms the ring-buffer batching argued in §3.4 and answers RQ4.

Throughput, however, does not tell a clean "io_uring wins" story: at 100 and 1,000 connections `epoll`'s mean throughput is marginally *higher* than `io_uring`'s, within the two engines' combined standard deviation. The strongest, most defensible claim from this data is architectural and syscall-based — io_uring drastically reduces user/kernel transitions — not a universal raw-throughput win over epoll on this small-payload, loopback, one-op-per-client workload.

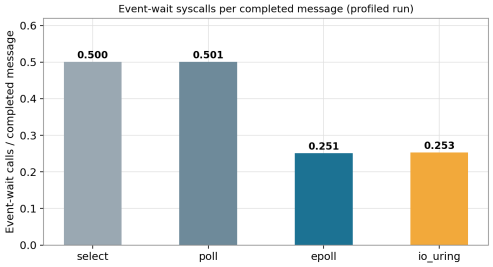
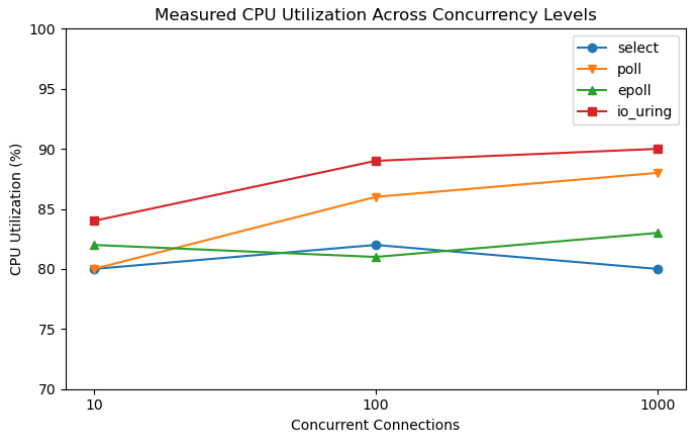


Figure 7: Event-wait calls per completed message (profiled run) — select/poll ≈ 0.50, epoll/io_uring ≈ 0.25.

7 CPU Utilization and Memory Scalability

The supplementary pass (§4) wraps each server in `/usr/bin/time -v` to obtain `Percent of CPU this job got` and `Maximum resident set size`, closing the CPU/RSS gap flagged in our previous submission. These are short, single-repetition bursts (10 msgs/connection) rather than the core benchmark's 5-repetition runs, so read them as indicative rather than as precision-matched to Table 3.



Engine	Peak RSS
select	1,040 KB
poll	1,040 KB
epoll	1,040 KB
io_uring	1,072 KB

Table 5: Peak RSS, constant across 10/100/1,000 conns.

Figure 8: Measured CPU utilization vs. connections.

CPU utilization rises with load for every engine (80–84% at 10 conns to 80–90% at 1,000), but `io_uring` is consistently the highest (84%→90%) and `select` the flattest (80%→80%, briefly touching 82% at 100) — consistent with io_uring keeping the CPU busier draining completion batches rather than blocking in a wait call, and select spending more of its time blocked in a single $O(N)$ syscall. Memory tells the cleanest story of the four metrics: all three readiness-based engines report an *identical* 1,040 KB peak RSS regardless of whether 10 or 1,000 connections are open, and `io_uring`'s 1,072 KB is only marginally higher (fixed-size SQ/CQ ring buffers allocated once at startup). None of the four engines' memory footprint scales with connection count in this implementation, since per-client state here is a small fixed-size struct indexed by fd, not a per-connection thread or buffer pool — so the classic "epoll/io_uring use less memory under load" argument does not show up as a *measured* effect here; it would require either far higher connection counts or a heavier per-client buffer to become visible.

8 Limitations

- **Percentiles are from a separate, smaller run**: the true p50/p90/p95/p99 (Fig. 5) come from a single-repetition, 10-msg/connection `asyncio` client at one concurrency level (1,000), not the 5-rep/100-msg core benchmark; treat them as tail-shape evidence, not a precision match to Table 3.
- **CPU/RSS are indicative, not averaged**: `/usr/bin/time -v` wraps a single short run per (engine, connection) pair; unlike Table 3's throughput, these are not averaged over 5 repetitions.
- **Loopback only**: results isolate server/event-loop behaviour from network variability but say nothing about a real NIC, WAN link, or packet loss.
- **select descriptor ceiling**: `FD_SETSIZE` is typically 1024, so the 1,000-connection point operates close to select's practical descriptor limit.
- **Profiling overhead**: profiling runs are 3–15× slower and used only for syscall/metric counts, never mixed into throughput/latency figures.

9 Conclusion

`select`/`poll` pay a repeated, $O(N)$-class descriptor-scanning cost on every wait call regardless of how many watched connections are active — the direct, measured cause of their throughput collapse and latency growth between 100 and 1,000 connections. `epoll` avoids scanning inactive descriptors via a persistent interest list (O(ready)); `io_uring` removes the per-operation syscall boundary via shared submission/completion ring buffers ($O(1)$ amortized), cutting normalized syscalls to just 0.101 per message (compared to ∼2.25–2.38 for the readiness-based models) — direct evidence that io_uring's ring-buffer design minimizes user→kernel transitions at the syscall boundary. The data does *not* support io_uring being unconditionally faster: epoll is occasionally, marginally ahead on raw throughput, and memory footprint here does not scale with connection count for any engine, since per-client state is a small fixed-size struct rather than a per-connection buffer or thread. Headline findings: (1) $O(N)$ scanning is the measured bottleneck behind select/poll's collapse; (2) io_uring's ring-buffer batching is a massive syscall-frequency win, not an unconditional throughput win; (3) CPU utilization rises modestly with load and asynchrony (io_uring highest, select flattest), while RSS stays load-independent across all four engines.

References

1. Kegel, D. "The C10K Problem." kegel.com.
2. `select(2)`, `poll(2)`, `epoll(7)`, `io_uring(7)` — Linux manual pages, man7.org.
3. Axboe, J. "Efficient IO with io_uring." kernel.dk.
4. liburing — userspace library for io_uring. GitHub: axboe/liburing.
5. `strace(1)`, `time(1)` — Linux manual pages, man7.org.